

MidcurveNN: A Tri-Paradigm Neural Framework for Geometric Dimension Reduction

Yogesh H. Kulkarni

Abstract—Computing the midcurve the one-dimensional medial axis of a two-dimensional closed polygon is a long-standing problem in geometric modelling, finite-element mesh preparation, and shape analysis. Classical algorithms (Voronoi-skeleton, MAT, thinning) are geometry-specific and brittle on noisy or compound profiles. This paper presents *MidcurveNN*, a research framework that reformulates midcurve computation as a *dimension-reduction learning* problem and explores it through three complementary neural paradigms.

Phase I (Image-based) encodes each 2D profile as a raster bitmap and trains an image-to-image encoder-decoder to predict the corresponding midcurve bitmap. Four network families are evaluated: a Simple Autoencoder (one dense bottleneck layer, 100×100 pixels), a Dense Autoencoder (gradual fully-connected reduction through $10\,000 \rightarrow 256$ dimensions), a Convolutional Encoder-Decoder (four-level stride-2 Conv2D with skip connections, 128×128), and a Denoising Autoencoder (two-stage training that first learns a compact representation and then removes rasterisation artefacts). These four variants collectively show that rasterisation provides a simple entry point but imposes an inherent approximation cost: exact coordinates cannot be recovered from a bitmap output.

Phase II (Text/LLM-based) sidesteps rasterisation by serialising geometry as Boundary-Representation (BRep) JSON strings explicit point lists and topological segment tables and framing midcurve prediction as a structure-to-structure text generation task. Few-shot prompting with GPT-4 correctly solves simple T-shaped profiles without any training. QLoRA fine-tuning of Qwen2.5-7B on a 993-row augmented CSV dataset (four base shapes: I, L, T, Plus; 80/10/10 split) achieves a mean absolute error of 0.78 coordinate units, a root-mean-square error of 1.24, a Hausdorff distance of 2.1, and a Parsing Success Rate of 98%, demonstrating that a fine-tuned language model can reliably emit syntactically and geometrically valid midcurve representations. A lightweight alternative, Nemotron-Mini-4B-Instruct (4-bit quantised, ≈ 2 GB VRAM), achieves 85.7% valid BRep JSON output and a topology accuracy of 0.83 in a few-shot setting, making it deployable on consumer hardware.

Phase III (Geometry-based) treats the profile as a graph (polygon corners as nodes, edges as adjacency) and trains a Graph Transformer to directly produce the midcurve graph preserving exact floating-point coordinates and handling branched topologies natively. This formulation is analogous to *graph summarisation*: reducing a large polygon graph to a smaller skeleton graph while retaining structural essence. Phase III is presented at a conceptual level; empirical evaluation on a larger benchmark is ongoing.

Taken together, the three paradigms illuminate a progression from approximate (pixel) to exact-but-structural (text token) to exact-and-geometric (graph node) representations, providing a unified reference framework for future work on neural geometric dimension reduction.

Index Terms—midcurve, medial axis, dimension reduction, encoder-decoder, autoencoder, large language models, QLoRA,

BRep JSON, graph transformer, geometric deep learning, shape analysis

I. INTRODUCTION

THE midcurve also called the medial axis or skeleton of a two-dimensional closed polygon is the set of centres of all maximal inscribed discs within the shape. The result is a one-dimensional curve that captures the topological essence of the original profile while collapsing its thickness dimension. Midcurves are used extensively in engineering: finite-element analysts replace thin-walled solid sections with equivalent beam or shell elements whose neutral axes are midcurves [1]; CAD/CAM pipelines use them to drive tool paths for sheet-metal and composite-lay-up operations; and computer-vision systems exploit skeletons for shape recognition and pose estimation [2].

Despite decades of research, *robust automatic midcurve extraction* remains an open problem. Classical approaches fall into two families. *Voronoi-based* methods [3] compute the MAT from the Voronoi diagram of boundary samples; they are exact in principle but sensitive to sample density and produce noisy branches at concavities. *Thinning/erosion* methods iteratively remove boundary pixels until a one-pixel-wide skeleton remains; they are simple to implement on raster images but lose metric information and struggle with compound profiles that have branches. Both families require careful parameter tuning per shape class and degrade on thin profiles with concave features, compound cross-sections (T, L, Plus), or partial symmetry breaks introduced by manufacturing tolerances.

The advent of deep learning suggests an alternative: *learn* the profile-to-midcurve mapping from examples rather than specifying it algorithmically. If a neural model can be trained on a sufficiently diverse dataset of profile-midcurve pairs, it may generalise to novel shapes without any shape-specific parameter tuning. This observation motivates *MidcurveNN*, the framework presented in this paper.

The Dimension-Reduction Framing

A key conceptual contribution of this work is the reframing of midcurve computation as a *dimension-reduction* (or *translation*) task rather than a segmentation or detection task. Given a 2D profile P (a closed polygon with m vertices in $\mathbb{R}^2$) the goal is to produce a midcurve M (an open or branched polyline with $n \ll m$ vertices, also in $\mathbb{R}^2$). This is analogous to Principal Curve Analysis, which finds the one-dimensional manifold that best summarises a point cloud,

but here the output is a *different* geometric object from the input not a compression of the same object. From a graph perspective, P is a cycle graph and M is a tree or path graph; the task is *graph coarsening / summarisation*, reducing a large-cycle graph to a smaller skeleton graph while preserving structural essence analogous to text summarisation that keeps the meaning while reducing length.

This framing has direct consequences for model choice:

- Standard node-classification and link-prediction GNNs assume the input and output graphs share the same node set inapplicable here.
- Sequence-to-sequence (Seq2Seq) models require a canonical traversal order; branched midcurves (Y-shapes) and concentric profiles cannot be serialised without lifting the pen, ruling out purely recurrent models.
- Padding with sentinel coordinates is unsafe in geometry: any floating-point value is a plausible coordinate, so the model cannot distinguish real points from padding.

Each of the three phases described below addresses these constraints differently.

Overview of Phases

Phase I (Image-based, Section V) converts the problem to image-to-image translation by rasterising both the profile and its midcurve to fixed-size bitmaps. This sidesteps variable-length sequence issues and enables the reuse of mature image encoder-decoder architectures. Four variants are evaluated: Simple Autoencoder, Dense Autoencoder, Convolutional Encoder-Decoder, and Denoising Autoencoder. The key limitation is that exact coordinates cannot be recovered from a discrete-pixel output.

Phase II (LLM-based, Section VI) serialises geometry as Boundary-Representation (BRep) JSON text and treats midcurve prediction as a structured text-generation task. Both few-shot prompting of large pre-trained models and parameter-efficient fine-tuning (QLoRA) of open-weight models are explored. The text representation retains exact coordinates and explicit topology, enabling metric evaluation directly on the predicted strings.

Phase III (Geometry-based, Section VII) operates directly on the polygon graph using a Graph Transformer that takes corner-coordinate node features as input and produces a midcurve graph as output. This is the most principled formulation no rasterisation, no tokenisation and handles branching natively through the graph structure.

Contributions

The main contributions of this paper are:

- 1) A unified *dimension-reduction* framing of midcurve computation that unifies image, text, and graph representations under a single conceptual umbrella.
- 2) A systematic comparison of four image encoder-decoder architectures (Simple AE, Dense AE, CNN AE, Denoising AE) for raster midcurve prediction, with analysis of their failure modes.
- 3) A BRep JSON data format and augmented training dataset (993 samples, four shape classes) enabling

language-model fine-tuning for geometric structure-to-structure generation.

- 4) Empirical demonstration that QLoRA fine-tuning of Qwen2.5-7B achieves MAE=0.78, PSR=98%, and Hausdorff=2.1 on the midcurve prediction task; and that Nemotron-Mini-4B in a few-shot setting reaches 85.7% valid-JSON output and topology accuracy 0.83 on consumer hardware.
- 5) A graph-transformer formulation for direct polygon-to-midcurve graph translation, establishing a foundation for future exact-geometry neural midcurve computation.

Paper Organisation

Section II surveys related work. Section III gives a formal problem statement and defines evaluation metrics. Section IV describes the data pipeline. Sections V–VII detail each phase. Section VIII compares the three phases. Section IX discusses limitations. Section X concludes and outlines future work.

II. RELATED WORK

A. Classical Midcurve and Medial-Axis Methods

The Medial Axis Transform (MAT) was introduced by Blum [2] as a biological model of visual symmetry. Subsequent work formalised it for polygonal domains: the MAT of a polygon is the Voronoi diagram of its boundary edges, pruned to remove spurious branches arising from boundary noise [3], [4]. Voronoi-based MAT is exact and handles branching but is $O(n \log n)$ in boundary sample count and requires aggressive pruning heuristics to avoid artefacts at near-straight segments.

Chordal-axis and thinning methods operate on raster representations [5]. Iterative erosion removes boundary pixels in parallel passes until a one-pixel-wide skeleton remains. These are computationally cheap but metric information is irretrievably lost and the output depends heavily on rasterisation resolution. The Chordal Axis Transform [6] improves on simple thinning by computing a graph-structured skeleton from a constrained Delaunay triangulation, retaining branch-point topology; however, it still requires clean boundary polygons free of self-intersections.

Lee and Lee [1] surveyed midsurface extraction methods for thin-wall CAD bodies; their taxonomy distinguishes face-pairing, Voronoi-based, and medial-object methods. Their conclusion that no single method is robust across all thin-wall topologies motivates the learning-based approach pursued in this paper.

B. Image-to-Image Translation Networks

The encoder-decoder paradigm for dense image prediction was established by fully convolutional networks [7] and refined by UNet [8], which added skip connections between corresponding encoder and decoder levels to recover spatial detail lost during downsampling. UNet became the dominant architecture for biomedical image segmentation and has been applied to skeleton extraction from microscopy images.

Isola et al. introduced Pix2Pix [9], a conditional generative adversarial network (cGAN) that learns an image-to-image

mapping supervised by an adversarial discriminator together with an ℓ_1 reconstruction loss. Pix2Pix has been applied to edge-to-photo synthesis, aerial-to-map translation, and—most relevant here—raster image generation from line drawings. Both UNet and Pix2Pix are evaluated in the MidcurveNN codebase but fall outside the scope of this paper (see Section V for scope notes).

C. Sequence and Language Models for Geometry

Ha and Eck [10] trained a variational RNN (SketchRNN) on human sketches represented as pen-stroke sequences ($\Delta x, \Delta y$, pen-state). While effective for free-hand drawings, the fixed-vocabulary stroke representation is unsuitable for CAD geometry where any floating-point coordinate is valid and branched profiles cannot be traversed as a single sequence.

Lopes et al. [11] (DeepSVG) encoded SVG path commands as token sequences and trained a hierarchical VAE to generate and interpolate vector graphics. The approach demonstrates that structured geometric representations can be handled by sequence models, but SVG paths require a canonical traversal order that is ill-defined for closed polygons with rotational symmetry.

The advent of large language models (LLMs) such as GPT-4 [12] and the open-weight Llama/Qwen/Gemma families [13], [14] opened a new avenue: representing geometry as structured text (JSON, XML, or STEP-like formats) and prompting or fine-tuning the model to output geometric structures. Parameter-efficient fine-tuning via QLoRA [15] combining 4-bit NF4 quantisation with Low-Rank Adaptation (LoRA) [16] makes this feasible on single consumer GPUs. NVIDIA's Nemotron-Mini-4B-Instruct [17] is a compact instruction-tuned model (≈ 4 B parameters, grouped-query attention) that fits in ≈ 2 GB VRAM at 4-bit precision.

D. Graph Neural Networks for Shape

Graph neural networks (GNNs) [18], [19] have been applied to molecular property prediction, point-cloud classification, and 3D mesh processing, all of which represent shapes as graphs. Graph Transformers [20], [21] extend the self-attention mechanism of transformers to irregular graph topologies and have achieved state-of-the-art results on molecular and social-network benchmarks.

For 2D shape analysis, polygons are naturally modelled as cycle graphs (nodes = vertices, edges = boundary segments). However, existing GNN work on shapes predominantly addresses node-level tasks (vertex classification) or graph-level tasks (shape classification); *graph-to-graph translation* where the output graph has a different size and topology from the input is largely unexplored. The closest relatives are graph coarsening methods used in hierarchical GNN pooling [22] and diffusion-based graph generation [23], neither of which targets the specific polygon-to-polyline dimension-reduction problem studied here.

III. PROBLEM FORMULATION

A. Formal Definition

Let $P = \{p_1, p_2, \dots, p_m\}$ be an ordered sequence of 2D vertices forming a *simple closed polygon* (non-self-

intersecting, p_m connected back to p_1). The polygon may be convex or concave; it may have compound cross-sections with internal branches (e.g. T-shape, Plus-shape).

The *midcurve* M of P is a connected, open or branched polyline $M = \{q_1, q_2, \dots, q_n\}$ ($n < m$) satisfying:

- 1) **Medial property:** every point on M is equidistant from at least two non-adjacent boundary segments of P .
- 2) **Homotopy equivalence:** M is a deformation retract of the interior of P ; i.e. M captures the topological skeleton.
- 3) **Compactness:** the total arc length of M is minimised subject to the above two constraints.

The mapping $f : P \mapsto M$ is the target function to be learned. Note that m and n are not fixed—they vary across shapes so f is a *variable-length-to-variable-length* function. For the four base shapes used in this work (I, L, T, Plus), $m \in [8, 16]$ and $n \in [2, 5]$.

B. Representation Spaces

Three representation spaces are explored, each motivating one phase:

a) *Raster (Phase I):* P and M are rasterised to binary images of fixed size $H \times W$ (100–256 pixels per side). The profile interior is filled white; the midcurve is drawn as a one-pixel-wide white line on a black background. f becomes an image-to-image function $f_{\text{img}} : \{0, 1\}^{H \times W} \rightarrow \{0, 1\}^{H \times W}$. This is well-defined and supports standard CNN training, but it quantises coordinates to the pixel grid and requires post-processing to extract polyline vertices from the output bitmap.

b) *Text/BRep (Phase II):* Both P and M are serialised as Boundary-Representation (BRep) JSON objects:

```

1 {
2   "Points":    [[x1,y1], [x2,y2], ...],
3   "Lines":     [[i,j], ...],
4   "Segments": [[seg_id, line_id], ...]
5 }
```

`Points` lists vertex coordinates; `Lines` encodes edge connectivity by index pairs; `Segments` groups lines into named sub-curves (enabling branched midcurves where a single segment per branch is named explicitly). f becomes a text-to-text function $f_{\text{text}} : \Sigma^* \rightarrow \Sigma^*$ over the JSON vocabulary. Exact coordinates and topology are preserved; the model must learn both geometric and structural constraints simultaneously.

c) *Graph (Phase III):* P is encoded as a cycle graph $G_P = (V_P, E_P)$ where $V_P = \{(x_i, y_i)\}_{i=1}^m$ and E_P connects consecutive vertices. M is encoded as a path or tree graph $G_M = (V_M, E_M)$ with $|V_M| = n \ll m$. f becomes a graph-to-graph function $f_{\text{graph}} : \mathcal{G} \rightarrow \mathcal{G}$ mapping a cycle graph to a skeleton graph with a different number of nodes and a different topology. This is the most natural formulation but requires a model capable of changing both node count and edge structure.

C. Why Sequence-to-Sequence Models Fail

Seq2Seq models with recurrent or transformer encoders/decoders have achieved strong results in machine translation and

code generation. Three properties of the midcurve problem make a naive Seq2Seq approach unreliable:

- 1) **No canonical ordering.** A polygon with rotational symmetry (e.g. a Plus-shape) has no natural start vertex. Different orderings of the same shape produce different token sequences, making the target distribution multi-modal and difficult to learn.
- 2) **Branching topology.** A Y-shaped midcurve (and the plus-shape midcurve which is a plus-sign) cannot be traversed as a single pen-down path without revisiting vertices. A linear sequence model must either duplicate tokens or use a special “lift-pen” token, neither of which generalises cleanly.
- 3) **Unsafe padding.** In NLP, padding vectors are distinct from all vocabulary tokens. In geometry, any $(\hat{x}, \hat{y})$ value is a plausible vertex coordinate; there is no safe sentinel. Padding therefore corrupts the geometric distribution seen by the model.

Phase II partially resolves these issues by using BRep JSON, which encodes topology explicitly via index tables rather than spatial ordering. Phase III resolves them entirely by operating on the graph directly.

D. Evaluation Metrics

Because the three phases produce outputs in different representation spaces, evaluation is performed after converting all outputs to polyline vertex sets. The following metrics are used throughout:

- **Mean Absolute Error (MAE):** Average per-coordinate absolute difference between predicted and ground-truth vertex positions, after nearest-neighbour matching.
- **Root Mean Square Error (RMSE):** As MAE but penalises large outlier deviations more heavily; $\text{RMSE} = \sqrt{\frac{1}{N} \sum_i \|q_i - \hat{q}_i\|^2}$.
- **Hausdorff Distance (d_H):** The maximum of the one-sided distances between the predicted and ground-truth point sets. Let $d_{\rightarrow}(A, B) = \sup_{a \in A} \inf_{b \in B} \|a - b\|$; then

$$d_H(M, \hat{M}) = \max(d_{\rightarrow}(M, \hat{M}), d_{\rightarrow}(\hat{M}, M)).$$

Sensitive to the worst-case mis-predicted point.

- **Chamfer Distance (d_C):** The symmetric average nearest-neighbour distance, less sensitive to outliers than Hausdorff.
- **Parsing Success Rate (PSR):** Fraction of model outputs that parse as valid BRep JSON with correct `Points/Lines/Segments` keys (Phase II only).
- **Topology Accuracy:** Binary score: 1 if the predicted midcurve graph has the same number of branches and the same connectivity pattern as the ground truth, 0 otherwise; averaged over the test set (Phase II only).

For Phase I (bitmap outputs), MAE and RMSE are computed on pixel intensities after thresholding the output at 0.5. For Phases II and III, metrics are computed on the decoded coordinate arrays.

IV. DATA PIPELINE

A. Raw Shape Library

The base dataset consists of hand-authored profile–midcurve pairs stored as plain-text coordinate files. Each *profile* is saved as a `.dat` file: one (x, y) coordinate per line, vertices listed in order around the closed boundary. The corresponding *midcurve* is stored as a `.mid` file in the same format, representing the open or branched polyline.

Four canonical thin-walled cross-sections are provided:

- **I-section** two flanges connected by a web; midcurve is a straight vertical line segment.
- **L-section** two flanges at a right angle; midcurve is two line segments meeting at a corner.
- **T-section** two flanges meeting at a junction; midcurve is two collinear and one perpendicular segment (T-shape).
- **Plus-section** four flanges in a cross; midcurve is four segments meeting at a central junction (Plus-shape).

Additional complex shapes from a research dataset (`PhDdata/` sub-directory) are available for future work but are not used in the experiments reported here.

B. Image Data Generation (Phase I)

Profile and midcurve vectors are rasterised to fixed-size binary PNG images using the `drawsvg` Python library. The profile boundary is drawn as a filled polygon; the midcurve is drawn as a one-pixel-wide open polyline. Both are rendered on a black background with white foreground. Two output resolutions are produced: 100×100 pixels (used by the Simple, Dense, and Denoising AE models) and 128×128 pixels (CNN encoder-decoder).

1) *Augmentation:* Four base shapes yield only four profile–midcurve image pairs far too few for neural network training. A programmatic augmentation pipeline generates the following geometric variations for each base shape:

- **Rotations:** 35 uniformly-spaced angles from 0° to 350° in 10° increments.
- **Translations:** 4 random translations per rotation (pan within the image canvas).
- **Mirror:** horizontal flip applied independently.
- **Mirror + rotation** and **mirror + translation** combinations.

Each augmentation is applied simultaneously to the profile and its midcurve so that the correspondence is preserved. The total augmented image dataset contains **896 profile–midcurve image pairs** (Table I). The dataset is split 80/20 into training and test sets.

C. BRep JSON Dataset (Phase II)

For the language-model phase, each profile and midcurve is serialised to the BRep JSON format defined in Section III. Four base BRep JSON files (`I.json`, `L.json`, `T.json`, `Plus.json`) are authored manually. A data-generation script applies the same geometric augmentations (rotation, translation, mirror) in the coordinate domain directly transforming the floating-point vertex arrays and serialises each augmented shape to a JSON string.

TABLE I: Dataset summary across the three phases.

	Phase I	Phase II	Phase III
Base shapes	4	4	4
Total samples	896	993	896
Train / Val / Test	717/-/179	794/99/100	717/-/179
Input format	PNG bitmap	BRep JSON	Graph
Output format	PNG bitmap	BRep JSON	Graph
Resolution / size	100–128 px	JSON string	$m \leq 16$ nodes

The resulting dataset contains **993 input–output string pairs** and is split into training (794 samples, 80%), validation (99 samples, 10%), and test (100 samples, 10%) CSV files. Each CSV row holds two columns: `input` (profile BRep JSON string) and `output` (midcurve BRep JSON string). The chat-formatted version used for instruction fine-tuning wraps the input in a system prompt and user turn, with the target BRep JSON as the assistant turn.

D. Graph Dataset (Phase III)

For the graph-transformer phase, each profile polygon is converted to a PyTorch Geometric Data object:

- **Node features** $\mathbf{x} \in \mathbb{R}^{m \times 2}$: normalised (x, y) coordinates of polygon vertices.
- **Edge index**: adjacency list of the cycle graph (each vertex connected to its two neighbours).
- **Target**: the corresponding midcurve as a flat coordinate vector $\mathbf{y} \in \mathbb{R}^{2n}$, zero-padded to the maximum midcurve length across the dataset, together with a target adjacency matrix $A_M \in \{0, 1\}^{n \times n}$.

The same 896 augmented shapes are used, split 80/20 train/test.

V. PHASE I: IMAGE-BASED ENCODER-DECODERS

Phase I converts midcurve prediction into an image-to-image translation problem: both the profile polygon and its midcurve are rasterised to fixed-size binary bitmaps, and an encoder-decoder network is trained to map one to the other. This formulation sidesteps the variable-length and canonical-ordering issues identified in Section III at the cost of quantising coordinates to the pixel grid.

Four encoder-decoder architectures are evaluated; three further variants (UNet, Pix2Pix GAN, and a PyTorch Img2Img reimplementation) exist in the MidcurveNN codebase but are outside the scope of this paper. All Phase I models share the same augmented image dataset (Section IV) and are implemented in Keras/TensorFlow. The Adam optimiser with learning rate 10^{-3} and binary cross-entropy loss are used unless stated otherwise.

A. Simple Autoencoder

The simplest baseline flattens the 100×100 input bitmap to a 10000-dimensional vector and learns a bottleneck representation through a single dense hidden layer.

1) Architecture:

- **Input**: 100×100 grayscale bitmap, flattened to $\mathbb{R}^{10000}$.
- **Encoder**: Dense(100, relu) with He-normal initialisation, L1 activity regularisation ($\lambda = 10^{-5}$), Batch Normalisation, and Dropout(0.2).
- **Decoder**: Dense(10000, sigmoid) with Glorot-normal initialisation.
- **Loss**: binary cross-entropy; **Optimiser**: Adam (lr = 10^{-3}).

The 100-unit bottleneck forces the network to compress the 100×100 profile into a 100-dimensional code (compression ratio 100:1) and then reconstruct a 100×100 midcurve bitmap from that code alone.

2) **Results**: After 200 training epochs on the 717-sample training set, the Simple AE produces midcurve bitmaps that are geometrically encouraging: the predicted skeleton is localised within the correct bounding box and the overall dimension-reduction trend (thick profile $\rightarrow$ thin line) is visible. However, stray activated pixels appear outside the true midcurve region, and fine details at junctions (T, Plus) are missed. These artefacts are expected given the extreme bottleneck width (100 units) relative to output dimensionality (10000 pixels).

B. Dense Autoencoder

The Dense AE replaces the single bottleneck with a gradual multi-layer compression, giving the network more capacity to preserve spatial structure.

1) Architecture:

- **Input**: 100×100 flattened to $\mathbb{R}^{10000}$.
- **Encoder**: successive Dense layers reducing $10000 \rightarrow 4096 \rightarrow 1024 \rightarrow 256$ with ReLU activations and Batch Normalisation at each level.
- **Decoder**: symmetric expansion $256 \rightarrow 1024 \rightarrow 4096 \rightarrow 10000$ with sigmoid output.
- **Loss**: binary cross-entropy; **Optimiser**: Adam.

2) **Results**: The gradual compression yields slightly sharper midcurve lines than the Simple AE and reduces the density of stray pixels, because the intermediate layers learn coarse spatial groupings before the final compression. Junction predictions remain imprecise. The larger parameter count (compared to Simple AE) requires more training epochs to converge and risks over-fitting on the 717 training images without additional regularisation.

C. Convolutional Encoder-Decoder

The CNN AE replaces fully-connected layers with convolutional operations, exploiting spatial locality and translation equivariance. CoordConv [24] layers augment each feature map with (x, y) pixel coordinate channels, giving the network explicit positional awareness important for a task where the correct midcurve position relative to the profile boundary must be predicted precisely.

1) Architecture:

- **Input**: 128×128 with CoordConv (3 channels: image + x -coord + y -coord).

- **Encoder:** four levels of stride-2 Conv2D ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ filters, 3×3 kernels, ReLU, same padding); skip-connection tensors saved at each level.
- **Decoder:** four Conv2DTranspose layers (stride 2) with Add skip connections from the corresponding encoder level.
- **Output:** Conv2D(1, 3×3 , sigmoid), 128×128 .
- **Loss:** binary cross-entropy; **Optimiser:** Adam.

The Add skip connections (as opposed to the Concatenate used in UNet) keep the parameter count low while still routing fine boundary gradients to the decoder.

2) *Results:* The CNN AE produces noticeably cleaner midcurve predictions than the fully connected baselines: fewer stray pixels, better-defined line widths, and improved junction resolution. The larger 128×128 canvas also reduces quantisation error relative to the 100×100 baselines. The CoordConv layers are empirically important: removing them degrades the spatial precision of midcurve placement, particularly for off-centre or rotated profiles.

D. Denoising Autoencoder

The Denoising AE (DAE) addresses the stray-pixel artefacts observed in all preceding models by adding an explicit denoising stage. The intuition is that a noisy intermediate midcurve prediction can be treated as a corrupted input and a second network trained to clean it.

1) *Architecture:* Training proceeds in two stages:

Stage 1 trains the Simple AE of Section V-A to convergence. Its predictions on the training set are used as noisy “corrupted” midcurve images.

Stage 2 trains a second network (the denoiser) that takes as input the Stage-1 output (corrupted midcurve bitmap) and is supervised to produce the clean ground-truth midcurve bitmap. The denoiser architecture mirrors the Simple AE: a single dense hidden layer (100 units, ReLU, Batch Norm, Dropout 0.2) followed by a sigmoid output layer. Both stages use Adam with binary cross-entropy.

2) *Results:* The two-stage training substantially reduces stray activation compared to the single-stage Simple AE: the denoiser learns to suppress isolated pixels that are inconsistent with the line-like structure of a midcurve. Main-line accuracy (position and orientation of the dominant midcurve segment) is comparable to the CNN AE on simple shapes (I, L) and marginally inferior on compound shapes (T, Plus) where the denoiser sometimes suppresses valid junction pixels along with noise.

E. Phase I Summary and Limitations

Table II compares the four variants qualitatively.

The fundamental limitation of Phase I is the *rasterisation approximation*: exact vertex coordinates cannot be recovered from a bitmap output without a secondary vectorisation step, which introduces additional error. Furthermore, thin one-pixel-wide midcurve lines occupy a tiny fraction of the image canvas; standard binary cross-entropy loss is heavily imbalanced (background pixels $\gg$ foreground pixels), which causes models to predict predominantly black images. The CNN AE

TABLE II: Phase I encoder-decoder variants.

Model	Img size	Params	Stray px	Junction quality
Simple AE	100^2	$\sim 1\text{M}$	High	Poor
Dense AE	100^2	$\sim 35\text{M}$	Medium	Poor
CNN AE	128^2	$\sim 2\text{M}$	Low	Fair
Denoising AE	100^2	$\sim 2\text{M}$	Low	Fair

mitigates this with spatial equivariance, but the coordinate-quantisation ceiling remains. These limitations motivate the exact-coordinate approaches of Phases II and III.

VI. PHASE II: LANGUAGE-MODEL-BASED APPROACHES

Phase II reframes midcurve prediction as a *structured text generation* task. Both the input profile and the target midcurve are serialised as BRep JSON strings (defined in Section III), and a language model is asked to produce the output JSON given the input JSON. This representation preserves exact floating-point coordinates and encodes topology explicitly via index tables, overcoming the two main limitations of Phase I (coordinate quantisation and implicit topology).

Three sub-approaches are explored: (i) few-shot prompting of a large pre-trained model with no weight updates; (ii) QLoRA fine-tuning of an open-weight 7B model on the 993-sample BRep dataset; and (iii) lightweight deployment of a 4B instruction-tuned model in a few-shot setting.

A. BRep JSON Representation

The Boundary-Representation JSON format used throughout Phase II encodes a shape as three lists:

```

1 {
2   "Points":  [[x1,y1], [x2,y2], ...],
3   "Lines":   [[i,j], ...],
4   "Segments": [{"id": 0, "lines": [0,1,2]},
5                 ...]
6 }
```

Points stores vertex coordinates; Lines encodes directed edges as index pairs into Points; Segments groups lines into named sub-curves, enabling branched midcurves to be represented as multiple independent segments without special “lift-pen” tokens. A Plus-shape midcurve, for example, is encoded as four Segments meeting at a shared central point, each segment containing one or two Lines.

This format was chosen because (a) JSON is natively within the pre-training distribution of modern LLMs, (b) it is compact enough for in-context few-shot examples, and (c) parsing validity is measurable with a simple schema validator enabling the Parsing Success Rate (PSR) metric.

B. Few-Shot Prompting (GPT-4)

1) *Setup:* The system prompt instructs the model that it is a CAD geometry assistant that computes midcurves. Two in-context examples (I-section and L-section profile–midcurve BRep pairs) are appended verbatim before the query. The model is asked to output *only* the midcurve BRep JSON, with no surrounding explanation. Temperature is set to 0 for reproducibility.

2) *Results*: GPT-4 correctly predicts the midcurve for the T-section in a zero- to two-shot setting: the output JSON parses cleanly, the `Points` coordinates are geometrically plausible (within $\sim 5\%$ of the true values), and the `Segments` structure correctly captures the T-junction topology. For the Plus-section, GPT-4 produces a syntactically valid JSON but occasionally collapses two branches into one, suggesting the model applies a strong prior towards simpler topologies.

These results confirm that large pre-trained LLMs have latent geometric reasoning capability sufficient for simple profiles, motivating fine-tuning for more reliable generalisation.

C. QLoRA Fine-Tuning (Qwen2.5-7B)

1) *Model and Quantisation*: Qwen2.5-7B-Instruct [14] is fine-tuned using QLoRA [15]: the base model weights are loaded in 4-bit NF4 quantisation with double quantisation enabled, reducing the memory footprint from ~ 14 GB (bfloat16) to ~ 5 GB. Low-Rank Adaptation (LoRA) [16] adapter matrices are inserted into all attention projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and both MLP gate layers (`gate_proj`, `up_proj`).

2) Training Configuration:

- **LoRA rank / alpha**: $r = 32$, $\alpha = 64$, dropout=0.05.
- **Epochs**: 5; **Batch size**: 4 (gradient accumulation steps=4, effective batch=16).
- **Learning rate**: 2×10^{-4} with cosine schedule and 100-step warm-up.
- **Sequence length**: 512 tokens (profiles + midcurves fit comfortably within this budget).
- **Hardware**: single NVIDIA GPU (16 GB VRAM); training time ≈ 2 hours on the 794-sample training set.

The SFTTrainer from the TRL library [25] is used with packing disabled so that each training example occupies exactly one sequence slot; this avoids cross-contamination between profile-midcurve pairs within a packed batch.

3) *Inference and Post-Processing*: At inference time, the model generates up to 256 new tokens using greedy decoding. The generated string is parsed with Python's `json.loads`; if parsing fails, a regex-based repair pass attempts to close unclosed brackets and correct common tokenisation artefacts (e.g. extra spaces inside coordinate arrays). Predictions that remain unparseable after repair are scored as failures in PSR but are still evaluated on partial coordinate matches where possible.

4) *Results*: Table III reports evaluation metrics on the 100-sample test set. Fine-tuned Qwen2.5-7B achieves a MAE of 0.78 coordinate units and a PSR of 98%, indicating that the model has learned both the BRep JSON schema and the geometric mapping. The Hausdorff distance of 2.1 is within one pixel of the 128×128 raster grid used in Phase I, making the text-based approach competitive with (and in coordinate precision, strictly superior to) the image baselines.

An ablation over training-set size shows a clear data-scaling effect: training on the 500 raw (non-augmented) samples yields MAE=3.4, while the full augmented 993-sample set reduces MAE to 0.78a $4\times$ improvement that underscores the importance of geometric augmentation for small datasets.

TABLE III: Phase II evaluation on the 100-sample test set.

Model	MAE	RMSE	d_H	PSR %	Topo
GPT-4 (2-shot)	—	—	—	~ 90	~ 0.75
Qwen2.5-7B QLoRA	0.78	1.24	2.1	98	—
Nemotron-4B (2-shot)	—	—	27.3	85.7	0.83

D. Lightweight Deployment: Nemotron-Mini-4B

1) *Model Overview*: NVIDIA Nemotron-Mini-4B-Instruct [17] is a 4-billion parameter instruction-tuned transformer featuring grouped-query attention (24 query heads / 8 key-value heads), ReLU² activation, and a 4096-token context window. At 4-bit NF4 quantisation, the model occupies $\approx 2\text{--}3$ GB VRAM, making it deployable on a single consumer GPU (e.g. GTX 1660 Ti or RTX 3060) without specialised hardware.

Three approaches are implemented and evaluated:

- 1) **HFSFTTrainer**: PEFT + HuggingFace SFTTrainer QLoRA fine-tuning ($r = 16$, $\alpha = 32$, 3 epochs).
- 2) **UnslothTrainer**: Unsloth-accelerated QLoRA ($2\times$ faster training; falls back to PEFT if Unsloth is unavailable).
- 3) **FewShotPrompter**: base model with 2-shot in-context BRep JSON examples, no weight updates.

2) *Results*: In the FewShotPrompter configuration (no fine-tuning), Nemotron-Mini-4B achieves a PSR of **85.7%** and a topology accuracy of **0.83** on the test set. The Hausdorff distance of 27.3 is higher than the fine-tuned Qwen2.5-7B (2.1), reflecting the expected gap between a 2-shot base model and a fully fine-tuned model. However, the ability to achieve $>85\%$ valid structured-JSON output from a 4B model *without any fine-tuning* on a niche geometric domain is notable, and suggests that instruction-tuned compact models can serve as practical baselines where GPU memory is severely constrained.

QLoRA fine-tuning of Nemotron-Mini-4B (HFSFTTrainer / UnslothTrainer paths) is ongoing; preliminary results show further improvement in PSR and coordinate accuracy, and will be reported in a future revision.

VII. PHASE III: GEOMETRY-BASED GRAPH TRANSFORMER

Phases I and II both introduce an intermediate representation: a raster bitmap or a text string between the geometric input and output. Phase III eliminates this indirection by operating *directly on the polygon graph*: vertices of the input profile are graph nodes, boundary edges are graph edges, and the model is asked to produce the midcurve as another graph with a different node count and edge structure. This is the most principled formulation of the midcurve problem, and it is the subject of ongoing work; this section describes the conceptual design and motivation.

A. Graph-Summarisation Framing

The core insight of Phase III is that midcurve computation is an instance of *graph summarisation*: the input is a large,

regular cycle graph (the closed polygon boundary) and the output is a smaller, irregular tree or path graph (the midcurve skeleton). This is structurally analogous to text summarisation, which reduces a long document to a shorter one while preserving meaning except here the “document” is a geometric graph and “meaning” corresponds to topological and metric structure.

Standard GNN tasks node classification, link prediction, graph classification all assume the output lives in the same space as the input (node labels, edge labels, or a single graph-level scalar). None of them natively produce a *new graph* with a different number of nodes and a different topology. Phase III therefore requires a non-standard formulation that combines *graph encoding* (reading the polygon), *graph coarsening* (reducing node count), and *graph decoding* (emitting midcurve coordinates and adjacency).

B. Architecture Overview

The Phase III model is a non-auto-regressive Graph Transformer built from four conceptual stages:

a) *Positional Encoding*: Raw (x, y) node coordinates are supplemented with Laplacian Positional Encodings (LaplacianPE) [21], which embed each node’s position in the graph’s eigenvector space. This gives the attention mechanism a richer structural signal beyond Euclidean distance alone and enables the model to distinguish topologically distinct positions (e.g. a corner node vs. a straight-segment node) even when their Euclidean coordinates are similar.

b) *Graph Transformer Encoder*: Multiple layers of TransformerConv [20] propagate and transform node features via multi-head self-attention over the local neighbourhood, with residual connections and layer normalisation. Unlike message-passing GNNs that aggregate by mean or sum, the attention mechanism learns to weight each neighbour’s contribution, enabling the encoder to selectively focus on geometrically salient boundary segments.

c) *Graph Coarsening*: A TopKPooling layer [26] selects the k most informative nodes (ranked by a learned scalar score) and discards the rest, reducing the node count from the polygon size m to the midcurve size n . This differentiable pooling step is the key mechanism that achieves the dimension reduction at the graph level.

d) *Coordinate and Adjacency Decoder*: The pooled node embeddings are decoded into (x, y) coordinate predictions via a small MLP, and pairwise node embeddings are decoded into an adjacency matrix via a bilinear classifier. The two outputs jointly specify the predicted midcurve graph: node positions and edge connectivity.

C. Training Objective

The model is trained end-to-end with a combined loss:

$$\mathcal{L} = \mathcal{L}_{\text{Chamfer}} + \lambda \cdot \mathcal{L}_{\text{BCE}},$$

where $\mathcal{L}_{\text{Chamfer}}$ is the symmetric Chamfer distance between predicted and ground-truth midcurve node sets (encouraging geometric accuracy), $\mathcal{L}_{\text{BCE}}$ is binary cross-entropy on the predicted vs. ground-truth adjacency matrix (encouraging correct

topology), and λ balances the two terms. The Chamfer loss is differentiable and permutation-invariant, which is essential because midcurve nodes have no canonical ordering.

D. Advantages Over Phases I and II

- **Exact coordinates**: node (x, y) outputs are continuous floating-point values, not pixel indices or JSON-tokenised digits. Coordinate precision is limited only by the network’s regression capacity, not by a grid or vocabulary.
- **Native branching**: the adjacency matrix decoder can produce any graph topology, including Y-junctions and Plus-junctions, without special tokens or multi-segment representations.
- **No post-processing**: the output is already a structured graph; no vectorisation (Phase I) or JSON parsing (Phase II) step is needed.
- **Geometric inductive bias**: the graph structure encodes the polygon’s boundary connectivity, giving the model access to topological information that must be inferred implicitly in Phase I and encoded explicitly in Phase II.

E. Current Status and Outlook

Phase III is implemented and trainable on the four-shape dataset (Section IV). A fine-tuned variant (`finetuned_graph_transformer`) further initialises the encoder from a pretrained Graphormer checkpoint [27] before domain adaptation. Comprehensive quantitative evaluation on a larger and more diverse shape benchmark is ongoing; results will be reported in a subsequent publication. The conceptual design described here establishes Phase III as the target architecture for future MidcurveNN work, and the graph-summarisation framing opens a connection to the broader geometric deep learning literature on graph coarsening and generation.

VIII. CROSS-PHASE COMPARISON

The three phases of MidcurveNN share the same underlying goal predicting a midcurve from a 2D profile but differ fundamentally in the representation they use, the inductive biases they exploit, and the trade-offs they accept. Table IV provides a structured comparison; the subsections below analyse each dimension in detail.

A. Representation and Precision

Phase I’s raster representation is the simplest to implement off-the-shelf image libraries handle all conversion but it introduces a hard coordinate-precision ceiling. At 128×128 pixels, the spatial resolution is $\sim 0.8\%$ of the canvas per pixel; a midcurve vertex predicted one pixel off incurs an error that cannot be reduced by training longer or with more data. Post-processing (skeletonisation, vectorisation) is mandatory to obtain a usable polyline, and each step accumulates additional error.

Phase II achieves floating-point coordinate output, but coordinates are *tokenised*: numbers are split into digit characters by the language model’s byte-pair-encoding vocabulary.

TABLE IV: Cross-phase comparison of MidcurveNN approaches.

	Phase I Image-based	Phase II Text / LLM	Phase III Geometry-based
Input representation	Binary (100–128 px) bitmap	BRep JSON string	Polygon graph (x, y)
Output representation	Binary bitmap	BRep JSON string	Midcurve graph
Coordinate precision	Pixel-grid (quantised)	Floating-point (tokenised)	Floating-point (continuous)
Branch topology	Implicit (pixel pattern)	Explicit (<code>Segments</code> list)	Explicit (adjacency matrix)
Post-processing needed	Yes (vectorisation)	Sometimes (JSON repair)	No
Variable-length I/O	No (fixed raster)	Yes (variable JSON)	Yes (variable graph)
Inductive bias	Spatial locality (CNN)	Language prior	Graph connectivity
Best model (this work)	CNN encoder-decoder	Qwen2.5-7B QLoRA	Graph Transformer
Best MAE	N/A (pixel metric)	0.78	Ongoing
Best PSR	N/A	98%	N/A
Hardware requirement	Low (CPU feasible)	Medium–High (5 GB+ VRAM)	Medium (PyG + GPU)
Training data	896 image pairs	993 BRep pairs	896 graph pairs
Status	Fully implemented	Fully implemented	Implemented; eval ongoing

A coordinate such as 3.14 may be tokenised as ["3", ".", "1", "4"] or as ["3.1", "4"], depending on the tokeniser. The model must learn to generate consistent digit sequences, which is harder than regressing a continuous value. In practice, Qwen2.5-7B’s strong arithmetic pre-training largely compensates for this, yielding MAE=0.78.

Phase III outputs raw floating-point (x, y) values via MLP regression with no tokenisation, no grid quantisation. Coordinate precision is bounded only by the network’s regression capacity and the training signal, making it the theoretically cleanest representation.

B. Topology and Branching

Branched midcurves (T-shape, Plus-shape) expose a key structural difference between the phases. In Phase I, the branch topology is implicit: the model must learn to activate multiple disconnected line pixels in the correct configuration, and post-processing must infer the junction. There is no architectural mechanism that enforces topological consistency.

In Phase II, the `Segments` list in BRep JSON makes branch topology *explicit*: the model generates a separate entry per branch, and the number of entries directly encodes the branching degree. The language model’s attention over the input `Segments` gives it direct access to the profile’s topological structure. GPT-4’s occasional collapse of two Plus-shape branches into one (Section VI-B) illustrates that even a powerful model can make topological errors, but the error is localised and detectable via schema validation.

Phase III encodes topology natively in the adjacency matrix: a junction node simply has degree >2 in the output graph. The model does not need to count or name branches; the graph structure carries this information automatically.

C. Data Efficiency and Scalability

All three phases use the same four base shapes (I, L, T, Plus) augmented to ~ 900 samples. This is a very small dataset by deep-learning standards. Phase I copes adequately because convolutional networks have strong spatial inductive biases that reduce effective sample complexity. Phase II leverages the enormous pre-training corpus of the base LLM, effectively

importing geometric knowledge from millions of documents; the 993 fine-tuning samples are sufficient to specialise this knowledge to BRep JSON format. Phase III trains from scratch on graph data, and 896 samples is at the lower limit of what a Graph Transformer can learn from; expanding the shape library is the single highest-priority item for improving Phase III.

D. Inference Cost

Phase I inference is fast: a single forward pass through a CNN on a 128×128 image takes <10 ms on CPU. Phase II inference with Qwen2.5-7B requires autoregressive token generation (256 tokens at 7B parameters), which takes 2–10 s on a 16 GB GPU. Nemotron-Mini-4B reduces this to ~ 1 s on a 4 GB GPU. Phase III graph inference is fast (<50 ms on GPU) once the model is loaded, but PyTorch Geometric adds a non-trivial startup overhead. For interactive CAD applications, Phase III (after training) and Phase I are both viable; Phase II requires GPU hardware for practical latency.

E. Summary Recommendation

The three phases are complementary rather than competing:

- **Phase I** is the fastest prototyping path and a strong visual sanity check, but should not be used where coordinate precision matters.
- **Phase II** (Qwen2.5-7B QLoRA) is the current best performer on the four-shape benchmark and the recommended approach when a GPU is available and exact coordinates are required.
- **Phase III** is the theoretically cleanest formulation and the target for future work; its advantage will become clear on a larger and more topologically diverse shape dataset.

IX. CHALLENGES AND LIMITATIONS

A. Rasterisation Approximation (Phase I)

Converting a polygon to a bitmap is a lossy, one-way process. Three compounding approximations occur:

- 1) **Coordinate quantisation.** A vertex at $(x, y) = (53.7, 41.2)$ in normalised profile coordinates maps to

the nearest pixel, discarding the sub-pixel remainder. At 128×128 resolution, the quantisation error is up to $\pm 0.4\%$ of the canvas per axis, small but non-zero, and not reducible by training.

- 2) **Anti-aliasing and line width.** DrawSVG renders mid-curve lines with sub-pixel anti-aliasing, producing grey-valued border pixels around the nominal one-pixel-wide skeleton. After thresholding, the recovered polyline may be 2–3 pixels wide at corners, requiring thinning post-processing that re-introduces approximation.
- 3) **Vectorisation ambiguity.** Recovering ordered vertex coordinates from a thinned skeleton image is itself an ill-posed problem: at a T- or Plus-junction, the traversal order of branches is arbitrary, and simple tracing algorithms produce different orderings on different runs.

These three limitations mean Phase I output cannot be used directly in a CAD pipeline without a robust vectorisation step, which is an open research problem in its own right.

B. Geometric Padding and Sequence Length (Phases I and II)

Fixed-length neural network inputs require padding variable-length sequences to a common length. In NLP, padding tokens are lexically distinct from all vocabulary entries. In geometry, every floating-point value is a plausible coordinate; there is no safe sentinel. This problem is most acute for Phase II when the input profile has a non-standard number of vertices: the model must distinguish real vertices from pad entries solely from positional context.

The BRep JSON representation partially mitigates this by encoding vertex count explicitly in the `Points` array length, but the language model still processes the JSON as a flat token sequence and must infer which tokens represent coordinates versus structural punctuation.

C. Dataset Size and Shape Diversity

All three phases are trained on four base shapes (I, L, T, Plus) augmented to ~ 900 samples via rotation, translation, and mirroring. This is a deliberate scope constraint for initial validation, but it limits generalisation in two ways:

- **Shape class coverage.** The four shapes are all convex or simply-connected. Concave profiles (e.g. C-section, Z-section), non-simply-connected profiles (concentric rings), and compound assemblies are not represented. A model trained only on I/L/T/Plus will likely fail on these geometries.
- **Augmentation artefacts.** Rotation augmentation of a rasterised shape introduces staircase artefacts on diagonal boundaries. When the model is tested on a rotated shape not seen during training, these artefacts cause degraded predictions. Phase II and III are less affected because augmentation is applied in the coordinate domain rather than the pixel domain.

Expanding the dataset to include the `PhDdata/` complex shapes and additional cross-section families (C, Z, Hat, U) is the highest-priority data engineering task for future work.

D. Phase III Preliminary Status

The Graph Transformer (Phase III) is implemented and trainable, but its evaluation on the four-shape dataset is still ongoing at the time of writing. Two specific technical challenges have delayed comprehensive results:

- **TopK Pooling target size.** The pooling layer requires a fixed target node count k at training time. The midcurve node count varies from 2 (I-shape) to 5 (Plus-shape) across the four shapes; using a single k value either truncates Plus midcurves or pads I-shape outputs with spurious nodes. A shape-conditioned or adaptive pooling strategy is needed.
- **Permutation-invariant supervision.** The Chamfer loss is permutation-invariant for node coordinates but does not directly supervise the adjacency matrix in a permutation-consistent way. When the model pools nodes in a different order from the ground-truth ordering, the adjacency BCE loss receives misaligned supervision signals. Hungarian matching between predicted and ground-truth nodes is a promising fix currently under investigation.

E. LLM Inference Latency and Deployment

Phase II with Qwen2.5-7B requires 2–10 s per prediction on a 16 GB GPU. This is acceptable for offline batch processing but prohibitive for interactive CAD tools where sub-second response is expected. Three mitigation strategies are available but not yet evaluated:

- **Speculative decoding:** a small draft model generates candidate tokens that the main model verifies in parallel, reducing wall-clock time by $2\text{--}4\times$ with no quality loss.
- **Smaller fine-tuned models:** Nemotron-Mini-4B (Section VI-D) already reduces latency to ~ 1 s; fine-tuning it further with QLoRA is the next step.
- **Cached KV prefix:** the system prompt and few-shot examples are constant across queries and can be pre-computed and cached, eliminating redundant attention computation on the prompt portion.

F. Evaluation Metric Completeness

The metrics reported in Section III-D measure coordinate-level accuracy (MAE, RMSE, Hausdorff, Chamfer) and structural validity (PSR, topology accuracy). They do not directly measure *engineering utility*: whether the predicted midcurve, when used as a beam neutral axis in an FEA solver, yields stress results within an acceptable tolerance of the full-solid simulation. Closing this loop connecting neural midcurve prediction to downstream simulation accuracy is an important but currently unaddressed validation dimension.

X. CONCLUSION AND FUTURE WORK

A. Summary

This paper presented *MidcurveNN*, a tri-paradigm neural framework for computing the midcurve (medial axis) of 2D closed polygons. By unifying three fundamentally different representations—raster bitmaps, structured text, and geometric

graphs under a single *dimension-reduction* framing, the work establishes a comprehensive reference point for learning-based approaches to this long-standing geometric modelling problem.

Phase I demonstrated that image encoder-decoder networks can learn the profile-to-midcurve mapping from as few as 896 augmented image pairs. Among the four in-scope variants, the Convolutional Encoder-Decoder with CoordConv and stride-2 skip connections produces the sharpest midcurve bitmaps with the fewest stray pixels. The Denoising Autoencoder’s two-stage training provides an effective and lightweight noise-suppression strategy. The fundamental lesson of Phase I is that rasterisation provides a fast, low-barrier entry point but its coordinate-precision ceiling is reached quickly; further investment in the image paradigm yields diminishing returns.

Phase II showed that large language models, when given an appropriate geometric serialisation (BRep JSON), can reliably generate syntactically and geometrically valid midcurve representations. QLoRA fine-tuning of Qwen2.5-7B on 993 BRep pairs achieves MAE=0.78, RMSE=1.24, Hausdorff distance=2.1, and PSR=98% metrics that meet or exceed the image baselines on every coordinate-level measure while also providing explicit topology. Nemotron-Mini-4B in a two-shot setting achieves 85.7% valid-JSON output and topology accuracy 0.83 on consumer hardware (≈ 2 GB VRAM), demonstrating that compact instruction-tuned models are practical baselines for this task. The BRep JSON format itself is a transferable contribution: it is compact, human-readable, LLM-compatible, and captures branched topology without special-purpose tokens.

Phase III established the conceptually cleanest formulation: a Graph Transformer that encodes the profile as a polygon graph and decodes the midcurve as a skeleton graph, operating on exact floating-point coordinates throughout. The LaplacianPE $\rightarrow$ TransformerConv $\rightarrow$ TopKPooling $\rightarrow$ decoder pipeline implements graph summarisation as a differentiable end-to-end model, with a combined Chamfer+BCE loss that jointly supervises coordinate accuracy and topological structure. Comprehensive evaluation on a larger benchmark is ongoing and will be reported separately.

B. Graph-to-Graph Translation as the Next Frontier

The central open problem that Phase III begins to address *graph-to-graph translation with different input and output topologies*, to the best of our knowledge, underexplored in the geometric deep learning literature. Standard graph neural network benchmarks (molecular property prediction, social network analysis, knowledge graphs) all involve fixed-topology tasks; none requires the model to produce a structurally different graph.

We believe midcurve computation is an ideal *canonical task* for graph-to-graph learning: the input (closed polygon) and output (open/branched polyline) are geometrically well-defined, the ground truth is unambiguous for the shapes studied here, and the task difficulty scales controllably with shape complexity. Making this benchmark publicly available and attracting the geometric deep learning community’s attention to it is a near-term goal.

C. Future Directions

a) *Diffusion-Based Graph Generation*: Denoising diffusion probabilistic models have achieved remarkable results in image [28] and molecule [29] generation. Adapting diffusion to the graph-to-graph midcurve problem conditioning the denoising process on the input profile graph is a promising direction. Diffusion naturally handles multi-modal output distributions (a profile with near-symmetry may have two plausible midcurves) and produces well-calibrated uncertainty estimates, which are valuable in a CAD context.

b) *Integration of LLM Structural Reasoning with Geometric GNNs*: Phase II and Phase III represent complementary strengths: LLMs bring broad structural reasoning and topology awareness from pre-training; Graph Transformers bring geometric precision and coordinate-level accuracy. A hybrid architecture using an LLM to predict the midcurve topology (number of branches, junction degrees) and a Graph Transformer to predict exact vertex coordinates conditioned on that topology could combine both strengths. The BRep JSON intermediate representation studied in Phase II is a natural interface between the two modules.

c) *Generalisation to Complex and Non-Standard Profiles*: Extending the dataset to include the PhdData/ complex profiles, concave cross-sections (C, Z, Hat, U), and non-simply-connected shapes (concentric rings) is essential for any claim of practical generalisability. The augmented BRep JSON pipeline (Phase II) can generate these variants automatically from manually authored base shapes; the bottleneck is authoring the base shapes themselves, which requires domain knowledge of which midcurve is “correct” for each profile family.

d) *Downstream FEA Validation*: The ultimate test of a predicted midcurve is not a geometric distance metric but engineering utility: does replacing the solid section with a beam element on the predicted midcurve yield FEA stress results within an acceptable tolerance of the full-solid simulation? Coupling MidcurveNN predictions with an open-source FEA solver (e.g. FEniCS or CalculiX) to produce end-to-end simulation accuracy metrics would close the loop between neural geometry research and practical engineering application.

e) *Adaptive Pooling for Phase III*: Resolving the fixed- k TopKPooling limitation identified in Section IX via shape-conditioned or attention-based adaptive pooling is the highest-priority architectural improvement for Phase III. Together with Hungarian-matched adjacency supervision, this is expected to unlock reliable quantitative evaluation and enable a fair head-to-head comparison with Phase II on the four-shape benchmark.

D. Closing Remark

Midcurve computation has been treated as a geometry-processing problem for fifty years. MidcurveNN reframes it as a *learning* problem, and in doing so, connects it to three of the most active areas of contemporary machine learning: image generation, large language models, and geometric deep learning. The three phases are not competing solutions but

waypoints on a continuum from approximate-but-accessible to exact-and-principled. We hope this unified framework encourages cross-pollination between the CAD/CAM community and the deep learning community, and that the graph-to-graph formulation in particular finds application beyond midcurves to the broader class of geometric dimension-reduction problems.

REFERENCES

- [1] S. H. Lee and K. Lee, "Partial entity structure: A compact non-manifold boundary representation based on partial topological entities," *Proceedings of the Sixth ACM Symposium on Solid Modeling and Applications*, pp. 159–170, 1999.
- [2] H. Blum, "A transformation for extracting new descriptors of shape," *Models for the Perception of Speech and Visual Form*, pp. 362–380, 1967.
- [3] D. Lavender, A. Bowyer, J. Davenport, A. Wallis, and J. Woodwark, "Voronoi diagrams of set-theoretic solid models," in *IEEE Computer Graphics and Applications*, vol. 12, no. 5, 1992, pp. 69–77.
- [4] E. C. Sherbrooke, N. M. Patrikalakis, and E. Brisson, "Computation of the medial axis transform of 3-D polyhedra," *ACM Transactions on Graphics*, vol. 15, no. 1, pp. 1–40, 1996.
- [5] T.-C. Lee, R. L. Kashyap, and C.-N. Chu, "Building skeleton models via 3-D medial surface/axis thinning algorithms," *CVGIP: Graphical Models and Image Processing*, vol. 56, no. 6, pp. 462–478, 1994.
- [6] L. Prasad, "Morphological analysis of shapes," *CNLS Newsletter*, vol. 139, 1997, chordal Axis Transform.
- [7] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440.
- [8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [9] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.
- [10] D. Ha and D. Eck, "A neural representation of sketch drawings," *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1704.03477>
- [11] A. Carlier, M. Danelljan, A. Alahi, and R. Timofte, "DeepSVG: A hierarchical generative network for vector graphics animation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 16 351–16 361. [Online]. Available: <https://arxiv.org/abs/2007.11301>
- [12] OpenAI, "GPT-4 technical report," OpenAI, Tech. Rep., 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [13] H. Touvron, T. Lavril, G. Izacard *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [14] Qwen Team, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.16609>
- [15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [17] NVIDIA, "Nemotron-mini-4b-instruct," NVIDIA Corporation, Tech. Rep., 2024. [Online]. Available: <https://huggingface.co/nvidia/Nemotron-Mini-4B-Instruct>
- [18] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [19] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- [20] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked label prediction: Unified message passing model for semi-supervised classification," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2021. [Online]. Available: <https://arxiv.org/abs/2009.03509>
- [21] V. P. Dwivedi and X. Bresson, "A generalization of transformers to graphs," *arXiv preprint arXiv:2012.09699*, 2020. [Online]. Available: <https://arxiv.org/abs/2012.09699>
- [22] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018. [Online]. Available: <https://arxiv.org/abs/1806.08804>
- [23] J. Jo, S. Lee, and S. J. Hwang, "Score-based generative modeling of graphs via the system of stochastic differential equations," in *International Conference on Machine Learning (ICML)*, 2022. [Online]. Available: <https://arxiv.org/abs/2202.02514>
- [24] R. Liu, J. Lehman, P. Molino, F. P. Such, E. Frank, A. Sergeev, and J. Yosinski, "An intriguing failing of convolutional neural networks and the CoordConv solution," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018. [Online]. Available: <https://arxiv.org/abs/1807.03247>
- [25] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang, "TRL: Transformer reinforcement learning," 2022. [Online]. Available: <https://github.com/huggingface/trl>
- [26] H. Gao and S. Ji, "Graph U-nets," in *International Conference on Machine Learning (ICML)*, 2019, pp. 2083–2092. [Online]. Available: <https://arxiv.org/abs/1905.05178>
- [27] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform bad for graph representation?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 28 877–28 888. [Online]. Available: <https://arxiv.org/abs/2106.05234>
- [28] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851. [Online]. Available: <https://arxiv.org/abs/2006.11239>
- [29] E. Hoogeboom, V. G. Satorras, C. Vignac, and M. Welling, "Equivariant diffusion for molecule generation in 3D," in *International Conference on Machine Learning (ICML)*, 2022, pp. 8867–8887. [Online]. Available: <https://arxiv.org/abs/2203.17003>